



Sign of the Times: Exploiting Poor Validation of AWS SNS SigningCertUrl

Eugene Lim @spaceraccoon

whoami

Eugene Lim

@spaceraccoon

Cybersecurity Group,
GovTech Singapore



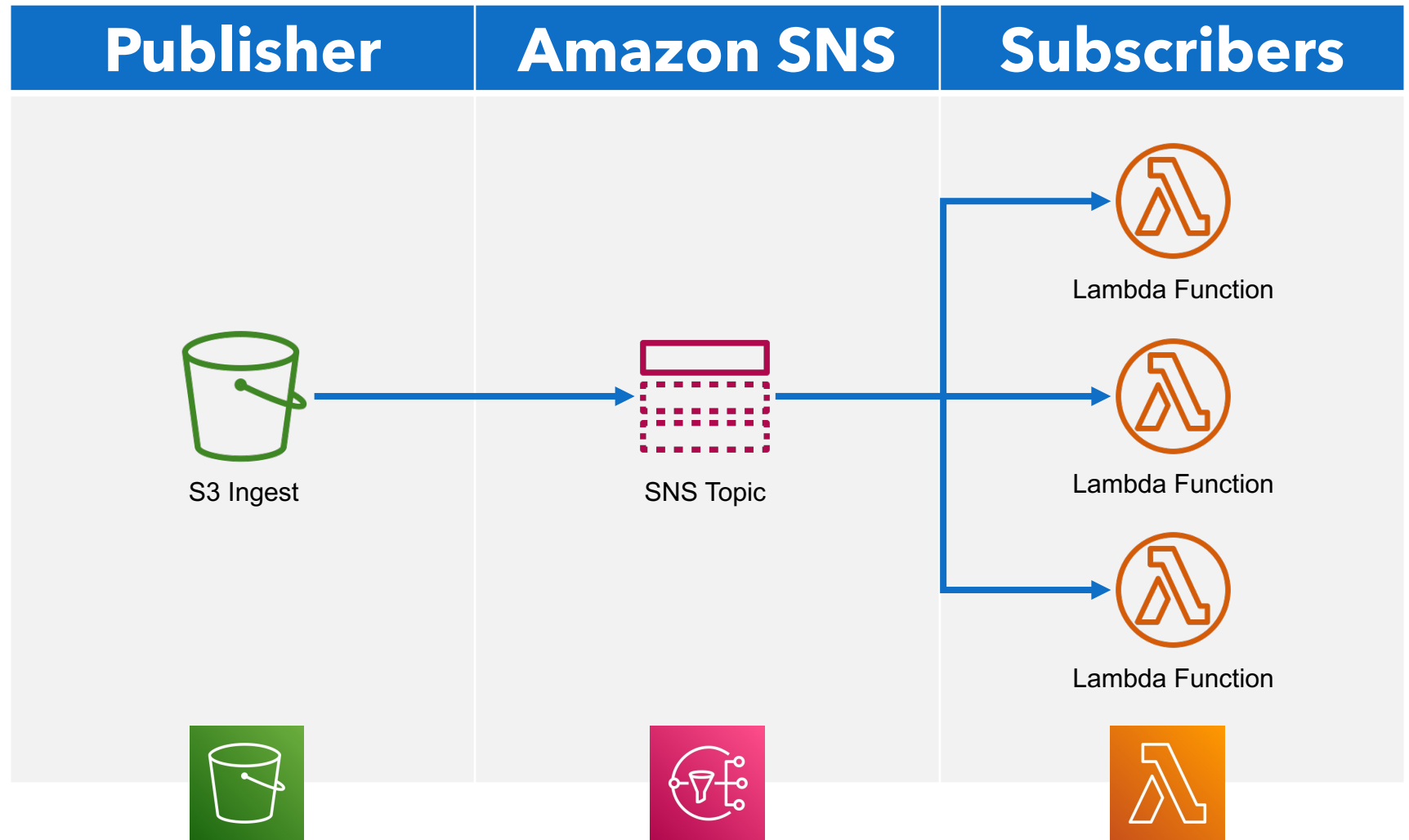
What is Amazon Simple Notification Service?



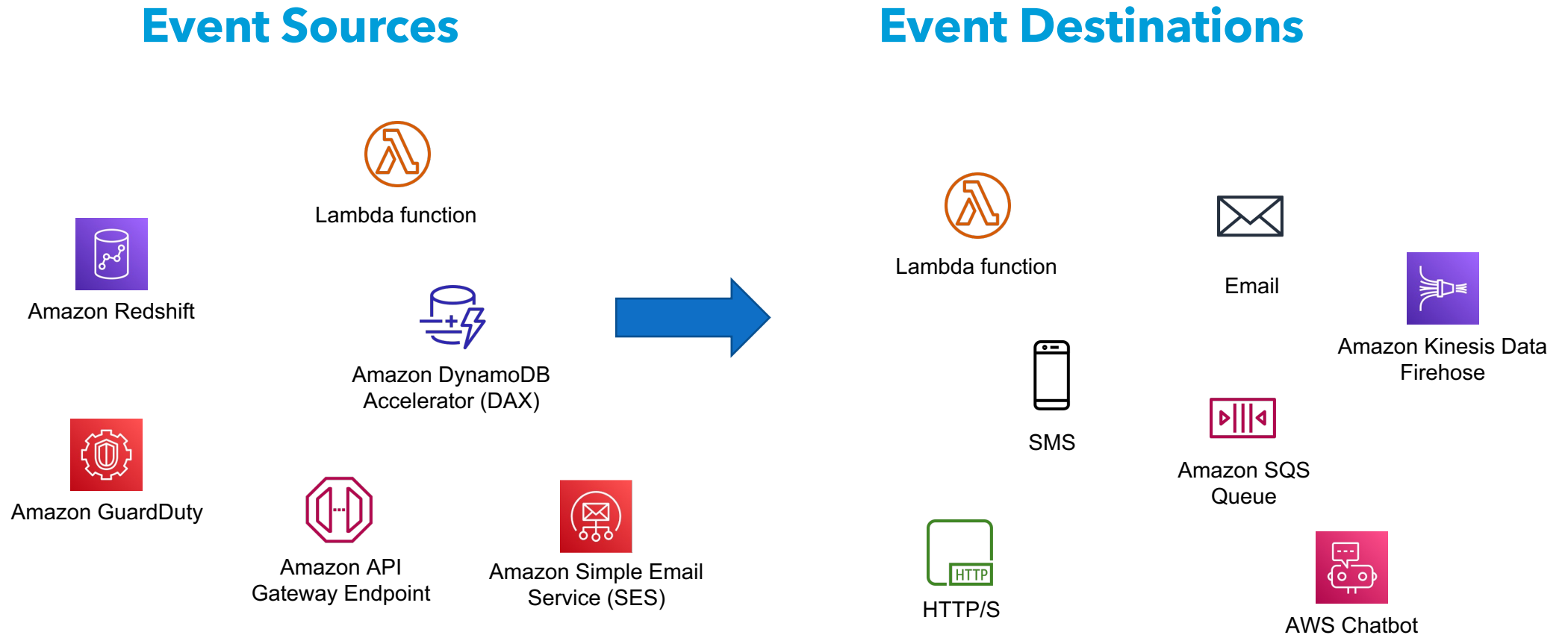
Just another publisher-subscriber message delivery service.*

*Queues sold separately.

Typical Amazon A2A SNS fanout pattern



Amazon SNS acts as the glue!*



*Not to be confused with AWS Glue.

Sample Amazon SNS HTTP message

```
POST / HTTP/1.1
x-amz-sns-message-type: Notification
x-amz-sns-message-id: 22b80b92-fdea-4c2c-8f9d-bdfb0c7bf324
x-amz-sns-topic-arn: arn:aws:sns:us-west-2:123456789012:MyTopic
x-amz-sns-subscription-arn: arn:aws:sns:us-west-2:123456789012:MyTopic:c9135db0-26c4-47ec-8998-413945fb5a96
Content-Length: 773
Content-Type: text/plain; charset=UTF-8
Host: example.com
Connection: Keep-Alive
User-Agent: Amazon Simple Notification Service Agent

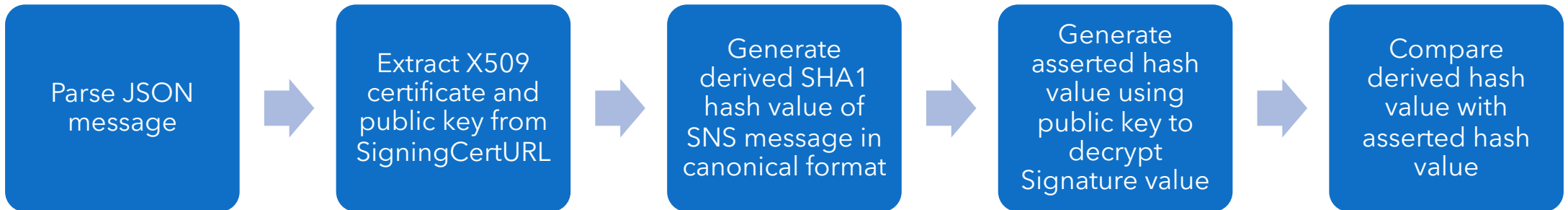
{
  "Type" : "Notification",
  "MessageId" : "22b80b92-fdea-4c2c-8f9d-bdfb0c7bf324",
  "TopicArn" : "arn:aws:sns:us-west-2:123456789012:MyTopic",
  "Subject" : "My First Message",
  "Message" : "Hello world!",
  "Timestamp" : "2012-05-02T00:54:06.655Z",
  "SignatureVersion" : "1",
  "Signature" : "EXAMPLEw6JRN...",
  "SigningCertURL" : "https://sns.us-west-2.amazonaws.com/SimpleNotificationService-
f3ecfb7224c7233fe7bb5f59f96de52f.pem",
  "UnsubscribeURL" : "https://sns.us-west-2.amazonaws.com/?Action=Unsubscribe&SubscriptionArn=arn:aws:sns:us-west-
2:123456789012:MyTopic:c9135db0-26c4-47ec-8998-413945fb5a96"
}
```

From Amazon Simple Notification Service Developer Guide

“Almost no one got JWT tokens and webhooks right on the first try. With webhooks, people almost always forgot to authenticate incoming requests...”

Ken Kantzer, “Learnings from 5 years of tech startup code audits”

Validating Amazon SNS messages?



```
"Signature" : "EXAMPLEw6JRN...",  
"SigningCertURL" : "https://sns.us-west-  
2.amazonaws.com/SimpleNotificationService-  
f3ecfb7224c7233fe7bb5f59f96de52f.pem",
```

```
Message  
My Test Message  
MessageId  
3d891288-136d-417f-bc05-901c108273ee  
SubscribeURL  
https://sns.us-east-2.amazonaws.com/?  
Action=...  
Timestamp  
2019-01-31T19:25:13.719Z  
Token  
233...  
TopicArn  
arn:aws:sns:us-east-2:123456789012:s4-MySNSTopic-1G1WEFCOXTCP  
Type  
SubscriptionConfirmation
```

But how to trust SigningCertURL?

Verifying the certificate given in SNS message SigningCertURL

Asked 5 years, 4 months ago Modified 5 years, 4 months ago Viewed 1k times



2



The [docs for SNS signature verification](#) mention that you need to download the cert given by `SigningCertURL` to check that the signature is valid.

What steps should I take to make sure the certificate there is valid (and that it doesn't belong to evil.example.com)?

The docs say "Verify the certificate was received from Amazon SNS". What needs to be checked specifically?

amazon-web-services

amazon-sns

Share Follow

asked Mar 8, 2017 at 3:45



djd

4,550

2

23

34

Add a comment

1 Answer

Sorted by:

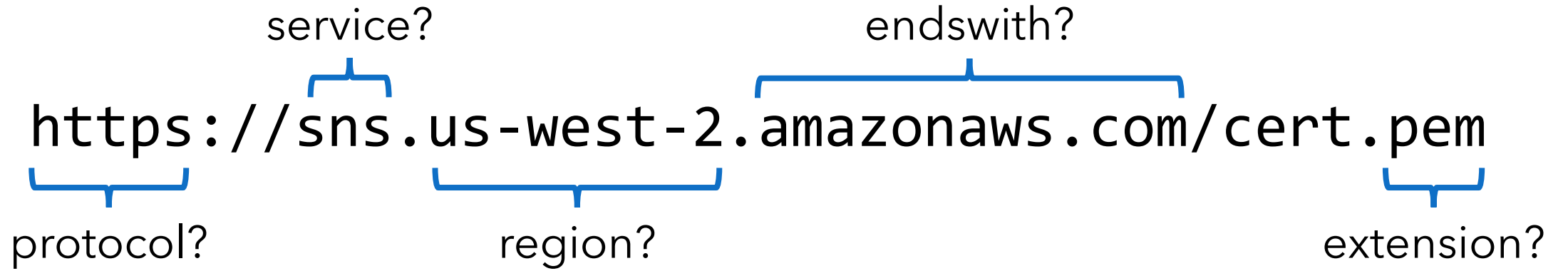
Highest score (default)



4

Verifying the source of the certificate means verifying that the hostname in the provided URL is a valid `sns.{region}.amazonaws.com` endpoint and then using an HTTPS user agent that doesn't have SSL certificate verification disabled.

Some custom validations used:*

The diagram shows the URL `https://sns.us-west-2.amazonaws.com/cert.pem` with blue brackets and labels indicating custom validation rules. The labels are: `protocol?` under `https://`, `service?` under `sns`, `region?` under `us-west-2`, `endswith?` under `amazonaws.com`, and `extension?` under `.pem`.

`https://sns.us-west-2.amazonaws.com/cert.pem`

protocol? service? region? endswith? extension?

*If they even validate it at all.

Enter Amazon S3.


service?

endswith?


protocol?

region?

extension?

`https://mysns.s3-us-west-2.amazonaws.com/evil.pem`

The diagram illustrates the components of the URL `https://mysns.s3-us-west-2.amazonaws.com/evil.pem`. Brackets and labels identify the following parts: `https` is the protocol; `mysns.s3` is the service; `us-west-2` is the region; `amazonaws.com` is the domain; and `evil.pem` is the extension. Each label is accompanied by a green checkmark icon.

Don't forget regexes ☹️

`https?://sns\.(.+)\.amazonaws\.com`

matches

`http://sns.evils3.amazonaws.com/evil.pem`

`sns.[a-z0-9\-.]+\s*.amazonaws.com`

matches

`snsthisismyamazonaws.com`

AWS SDK official regex:

PENDING DISCLOSURE APPROVAL BY AWS

```
^sns\[a-zA-Z0-9\-\]{3,}\.amazonaws\.com(\.cn)?$
```

Used by official AWS SDKs

Only matching S3 bucket:

PENDING DISCLOSURE APPROVAL BY AWS

```
^sns\[a-zA-Z0-9\-\]{3,}\.amazonaws\.com(\.cn)?$
```

Matches minimum 3 characters; naming
convention since removed by AWS


sns.s3-us-west-2.amazonaws.com

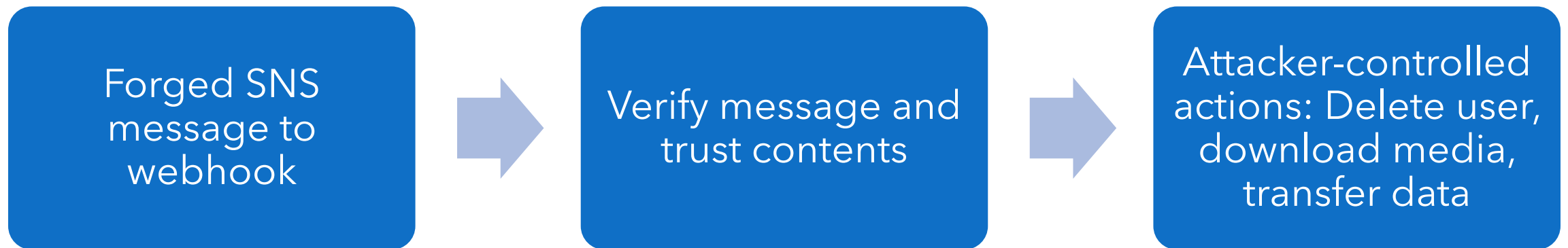
Publicly writable!!!

Bug bounty timeline

PENDING DISCLOSURE APPROVAL BY AWS

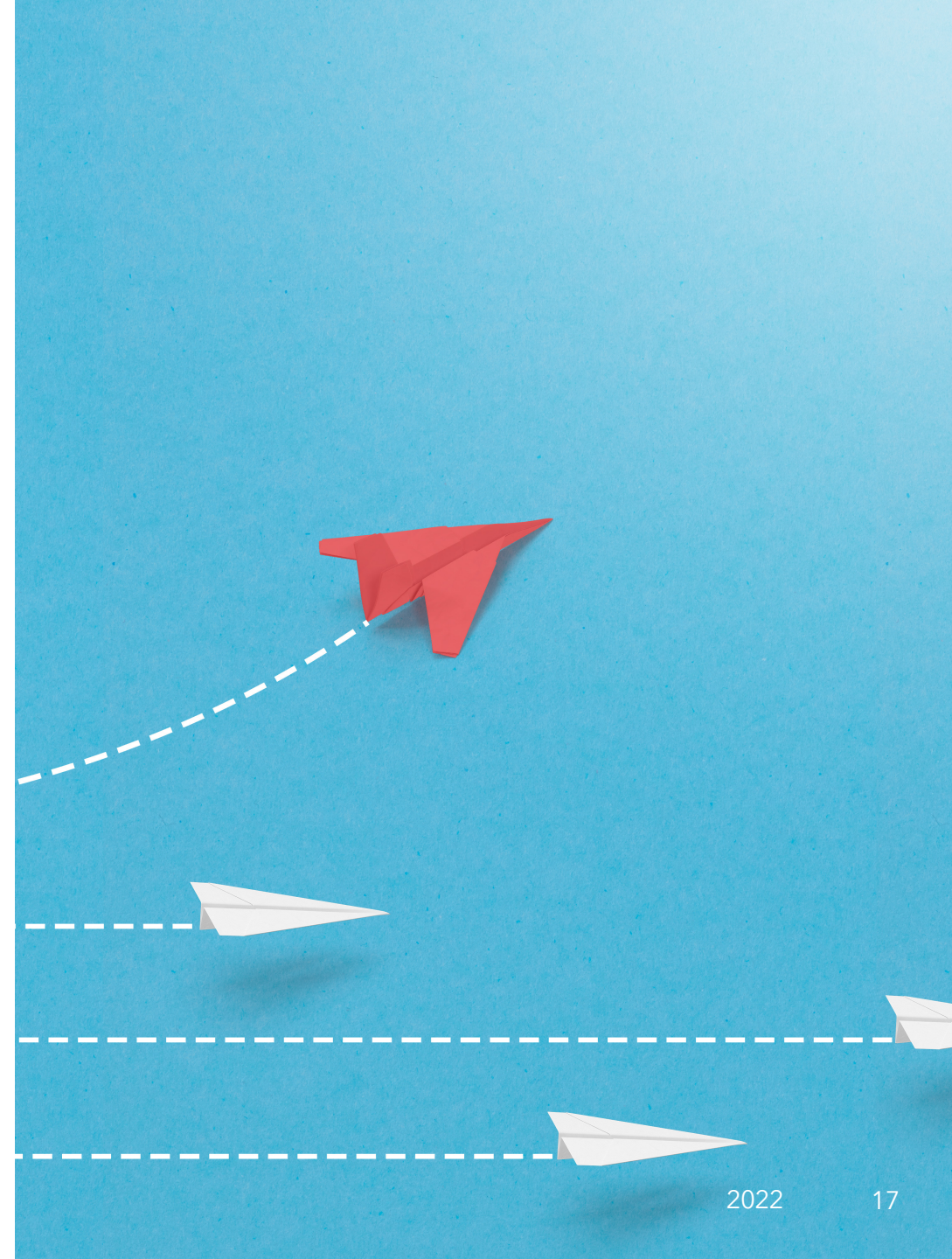
- 2022-06-05 07:55:02 +0000: Reported
- 2022-06-05 13:25:23 +0000: Triaged
- 2022-07-08 17:56:45 +0000: Bounty awarded; initial patch to remove s3-<REGION>.amazonaws.com naming convention to block regex bypass while maintaining backward compatibility with SDKs

What could go wrong?



Summary

Public webhooks suffer from the perennial problem of verifying messages; poor implementations of SNS verification pattern leads to more cloud shenanigans!



Thank you

Eugene Lim

@spaceracconsec

spaceraccoon.dev

